

Banco de preguntas de Programación Web

400 preguntas de opción múltiple sobre los apuntes 02 a 22. Marcá una sola opción en cada pregunta.

Nombre _____

Legajo _____

01	Variables, alcance y expresiones	1–13	06	Conversión y coerción de tipos	72–86	11	Estructuras de control	163–182	17	Gestión de archivos con Node.js	282–312
02	El tipo <code>boolean</code>	14–28	07	El tipo <code>symbol</code> y los protocolos de JavaScript	87–101	12	Errores y excepciones	183–200	18	Desarrollo web: del documento a una aplicación del clima	313–362
03	Los tipos <code>number</code> y <code>bigint</code>	29–43	08	Arrays y matrices	102–125	13	Funciones	201–226	19	Arrow.js: del contador al inventario	363–374
04	El tipo <code>string</code> y Unicode	44–58	09	Objetos, propiedades y referencias	126–147	14	Programación funcional y pipelines de datos	227–246	20	Cómo consumir una API REST	375–400
05	<code>undefined</code> , <code>null</code> y la ausencia de datos	59–71	10	<code>Map</code> , <code>Set</code> y colecciones especializadas	148–162	15	Recursividad y árboles binarios	247–268			
						16	Programación Asíncrona	269–281			

01 Variables, alcance y expresiones 1–13

- 1 ¿Qué nombre de variable describe mejor la cantidad de días que faltan para un vencimiento?

a

diasHastaVencimiento

b

fechaDeVencimiento

c

diasTranscurridosDelPlazo
- 2 Después de ejecutar `let total;`, ¿qué valor tiene `total`?

a

null

b

ReferenceError

c

undefined
- 3 ¿Qué hace la sentencia `const iva = 0.21;`?

a

Declara iva sin valor

b

Declara e inicializa iva

c

Reasigna una constante anterior
- 4 Una variable acumula un total que cambia en cada paso de un cálculo. ¿Con qué palabra conviene declararla?

a

const

b

class

c

let
- 5 Después de `const pedido = { estado: "nuevo" };`, ¿qué operación está permitida?

a

Reasignar `pedido` a un objeto nuevo

b

Modificar `pedido.estado`, porque `const` fija solo la variable

c

Nada: el objeto queda inmutable
- 6 Una variable se declara con `var` dentro de un `if`, y ese `if` está dentro de una función. ¿Hasta dónde llega el alcance de la variable?

a

A toda la función

b

Solo al bloque `if`

c

Al módulo que contiene la función
- 7 ¿Qué formas de declarar variables quedan limitadas al bloque de llaves de un `if`?

a

`let` y `const`

b

`var` y `let`

c

`var`, `let` y `const`

- 8 Fuera de un bloque existe una variable `estado` con el valor `"global"`. Dentro del bloque se declara `const estado = "local"`. ¿Qué valor tiene `estado` dentro del bloque?

a

El estado global

b

undefined

c

El estado local
- 9 Dentro de un bloque, se lee una variable antes de la línea `const` que la declara. ¿Qué ocurre?

a

Se obtiene undefined

b

Se produce ReferenceError

c

Se obtiene su valor futuro
- 10 ¿Qué muestra en la consola este código?

```
console.log(total);  
var total = 100;
```

a

undefined

b

100

c

ReferenceError
- 11 Una función exterior declara una variable local y devuelve una función interior que usa esa variable. ¿Qué pasa con la variable cuando la función exterior termina?

a

El dato se libera al terminar la primera función

b

La función devuelta recibe una copia congelada del dato

c

La función devuelta sigue accediendo a esa variable local
- 12 ¿Cuál de estos fragmentos es una expresión, es decir, produce un valor?

a

`if (edad >= 18) {}`

b

`edad >= 18`

c

`const edad = 18;`
- 13 Después de ejecutar `let n = 5; const previo = n++;`, ¿cuánto valen `previo` y `n`?

a

previo vale 5 y n vale 6

b

previo vale 6 y n vale 6

c

previo vale 5 y n vale 5

02 El tipo boolean

14-28

- 14 ¿Cuáles son los únicos valores del tipo boolean?
- (a) 1 y 0
 - (b) truthy y falsy
 - (c) true y false
-
- 15 ¿Puede un string no vacío usarse directamente como condición en un `if`?
- (a) No, se exige un booleano literal
 - (b) Sí, se interpreta como truthy
 - (c) Sí, pero se interpreta como false
-
- 16 ¿Cuál de estos valores es falsy?
- (a) `" "`
 - (b) `"false"`
 - (c) `NaN`
-
- 17 ¿Qué devuelve `Boolean({})`?
- (a) false
 - (b) true
 - (c) undefined
-
- 18 ¿Qué devuelve `Boolean("false")`?
- (a) true, porque el string no está vacío
 - (b) false, porque el texto dice false
 - (c) Lanza `TypeError`, porque no es un booleano válido
-
- 19 ¿Qué expresión convierte `valor` a booleano de la forma más explícita y clara?
- (a) `valor == true`
 - (b) `Boolean(valor)`
 - (c) `new Boolean(valor)`
-
- 20 ¿Qué devuelve `!0`?
- (a) false
 - (b) 0
 - (c) true
-
- 21 ¿Qué devuelve `"" && ejecutar()`?
- (a) El string vacío sin llamar a ejecutar
 - (b) El resultado de ejecutar
 - (c) El booleano false después de llamar a ejecutar
-
- 22 ¿Qué devuelve `"Ana" && 20`?
- (a) "Ana"
 - (b) 20
 - (c) true
-
- 23 ¿Qué devuelve `0 || false || null`?
- (a) false
 - (b) true
 - (c) null
-
- 24 ¿Qué devuelve `0 ?? 1`?
- (a) 0
 - (b) 1
 - (c) null

25 ¿Qué devuelve `5 === "5"`?

- (a) false
- (b) true
- (c) undefined

26 ¿Qué expresión comprueba correctamente si `valor` es `NaN`?

- (a) `valor === NaN`
- (b) `typeof valor === "NaN"`
- (c) `Number.isNaN(valor)`

27 ¿Qué expresión equivale a `!(a && b)`?

- (a) `!a && !b`
- (b) `!a || !b`
- (c) `a || b`

28 ¿Cómo evalúa JavaScript la expresión `esAdmin || esEditor && estaActivo`?

- (a) `(esAdmin || esEditor) && estaActivo`
- (b) `esAdmin || (esEditor && estaActivo)`
- (c) `!(esAdmin || esEditor) && estaActivo`

03 Los tipos number y bigint

29-43

29 ¿Qué tipo representa tanto 42 como 3.5 en JavaScript?

- (a) number
- (b) bigint
- (c) integer

30 ¿Qué valor decimal representa `0b101010`?

- (a) 101010
- (b) 52
- (c) 42

31 ¿Qué valor tiene `46_000_000`?

- (a) 46000
- (b) `SyntaxError`
- (c) 46000000

32 ¿Qué devuelve `-5 % 3` en JavaScript?

- (a) -2
- (b) 1
- (c) NaN

33 ¿Qué resultado da `1 / 0` en JavaScript?

- (a) Una excepción de división
- (b) Infinity
- (c) NaN

34 ¿Qué produce `0 / 0`?

- (a) Infinity
- (b) NaN
- (c) 0

35 ¿Por qué `Number.isNaN("hola")` es más confiable que `isNaN("hola")` para detectar `NaN`?

- (a) Convierte el string a número antes de comprobar
- (b) Devuelve true para cualquier string no numérico
- (c) No convierte el string a número antes de comprobar si es NaN

36 ¿Qué devuelve `Object.is(0, -0)`?

- a false
- b true
- c TypeError

37 ¿Por qué `0.1 + 0.2` puede no ser exactamente 0.3?

- a Number guarda los decimales con solo dos cifras
- b El motor redondea el resultado a 16 decimales
- c Muchos decimales no tienen representación exacta en binario

38 ¿Qué comprueba `Number.isSafeInteger(valor)`?

- a Que es un número finito y no NaN
- b Que es un entero representable sin pérdida
- c Que el valor entra en un entero de 32 bits

39 ¿Qué devuelve `Number("42px")`?

- a NaN
- b 42
- c "42"

40 ¿Qué devuelve `parseInt("42px", 10)`?

- a NaN
- b 42
- c 0

41 ¿Qué valores dan `Math.floor(-3.2)` y `Math.trunc(-3.2)`?

- a -3 y -4
- b -3 y -3
- c -4 y -3

42 ¿Qué tipo devuelve `(12.5).toFixed(2)`?

- a string
- b number
- c bigint

43 ¿Qué ocurre con `10n + 2`?

- a Devuelve 12n
- b Devuelve 12 como number
- c Lanza TypeError

04 El tipo string y Unicode

44-58

44 ¿Qué comillas permiten insertar expresiones dentro de una cadena?

- a Las comillas invertidas, con la forma `${expresión}`
- b Las comillas simples y dobles
- c Las comillas dobles con `${}`

45 ¿Qué representa `\n` dentro de una cadena JavaScript?

- a Una barra invertida seguida de una n
- b Un salto de línea
- c El final de la cadena

46 Después de `const s = "casa"; s[0] = "C";`, ¿qué valor tiene `s`?

- a "Casa"
- b "casa"
- c "C"

47 Si se llama a `original.trim()` sin guardar el resultado, ¿qué pasa con el string `original`?

- a Pierde los espacios en el mismo string
- b Cambia solo si tenía espacios en los extremos
- c No cambia; trim devuelve una cadena nueva sin los espacios

48 ¿Qué devuelve `"JavaScript".at(-1)`?

- a "t"
- b "J"
- c undefined

49 ¿Qué devuelve `"JavaScript".slice(0, 4)`?

- a "JavaS"
- b "Java"
- c "Script"

50 ¿Qué método de string responde directamente si una frase contiene un texto dado?

- a split
- b startsWith
- c includes

51 ¿Qué devuelve `"uno dos uno".replaceAll("uno", "1")`?

- a "1 dos 1"
- b "1 dos uno"
- c "uno dos 1"

52 ¿Qué devuelve `"a,b,c".split(",")`?

- a La cadena "abc"
- b Un array con una sola cadena "a,b,c"
- c Un array con "a", "b" y "c"

53 ¿Qué expresión convierte `valor` en texto sin fallar aunque `valor` sea `null`?

- a `valor.toString()`
- b `String(valor)`
- c `valor.join()`

54 ¿Qué valor tiene `"😊".length`?

- a 2 unidades UTF-16
- b 1, porque es un solo carácter visible
- c 4 puntos de código

55 ¿Qué produce `Array.from("A😊B")`?

- a ["A", dos mitades de emoji, "B"]
- b ["A😊B"]
- c ["A", "😊", "B"]

56 ¿Qué herramienta divide un texto en caracteres tal como los ve una persona (grafemas), incluidos los emoji compuestos?

- a `Array.from` sobre la cadena
- b `Intl.Segmenter`
- c `normalize` con la forma NFC

57 Dos cadenas se ven iguales, pero están escritas con distintas secuencias Unicode, por ejemplo «é» como un solo carácter o como «e» más un acento. ¿Qué hay que hacer para poder compararlas?

- a Normalizar ambas a NFC
- b Aplicar trim a una sola
- c Comparar sus longitudes UTF-16

- 58 ¿Qué herramienta ordena alfabéticamente nombres en español respetando la ñ y los acentos?
- (a) Intl.Collator configurado para es
 - (b) El operador menor que sin configuración
 - (c) Comparar la longitud de los strings

05 undefined, null y la ausencia de datos

59-71

- 59 ¿Qué valor tiene pendiente tras let pendiente; ?
- (a) null
 - (b) "" (cadena vacía)
 - (c) undefined
- 60 ¿Qué valor se asigna a propósito para indicar que un dato no existe o no tiene valor?
- (a) undefined
 - (b) null
 - (c) NaN
- 61 ¿Qué devuelve typeof null ?
- (a) "null"
 - (b) "undefined"
 - (c) "object"
- 62 En un programa, cantidad puede valer 0 legítimamente. ¿Por qué if (!cantidad) no sirve para detectar que falta el dato?
- (a) Porque también entra cuando cantidad vale 0, que es válido
 - (b) Porque no entra cuando cantidad es undefined
 - (c) Porque !cantidad lanza error si falta el dato
- 63 ¿Qué devuelve false ?? true ?
- (a) true
 - (b) false
 - (c) undefined
- 64 Si usuario es null, ¿qué produce usuario?.direccion?.ciudad ?
- (a) null
 - (b) undefined
 - (c) TypeError
- 65 Con function f(x = 3) { return x; }, ¿cuándo se aplica el 3?
- (a) Cuando el argumento es null
 - (b) Cuando el argumento es falsy
 - (c) Cuando falta o es undefined
- 66 ¿Qué diferencia hay entre los objetos {} y { valor: undefined } ?
- (a) Solo el segundo tiene la propiedad valor como propia
 - (b) Leer .valor da undefined solo en el primero
 - (c) JSON.stringify los convierte en textos distintos
- 67 ¿Qué diferencia hay entre [1, , 3] y [1, undefined, 3] en la posición 1?
- (a) Solo el segundo tiene esa posición presente
 - (b) Solo el primero tiene esa posición presente
 - (c) Ambos tienen exactamente la misma estructura

- 68 ¿Qué devuelve [10, 20].indexOf(50) si no encuentra el valor?
- (a) undefined
 - (b) null
 - (c) -1

- 69 ¿Qué ocurre con una propiedad cuyo valor es undefined al serializar un objeto con JSON.stringify?
- (a) Se escribe como null
 - (b) Se omite
 - (c) Se escribe como "undefined"

- 70 Una función devuelve null en tres situaciones distintas: todavía no hay datos, la búsqueda no encontró nada o hubo un error. ¿Qué representación distingue claramente cada caso?
- (a) Un objeto con una propiedad estado que nombre cada caso
 - (b) Un booleano que indique si hubo resultado
 - (c) Distinguir null de undefined para cada caso

- 71 ¿Qué se gana al normalizar (limpiar y dar un formato único a) los datos externos apenas se reciben?
- (a) La entrada deja de requerir validación posterior
 - (b) Todo el programa usa datos con la misma forma
 - (c) JSON.stringify conserva los campos undefined

06 Conversión y coerción de tipos

72-86

- 72 ¿Qué expresión convierte textoCantidad a número de forma explícita?
- (a) textoCantidad * 1
 - (b) textoCantidad.valueOf()
 - (c) Number(textoCantidad)
- 73 ¿Por qué "6" - 1 produce 5?
- (a) La resta solo opera con números y convierte el string a número
 - (b) La resta concatena y luego convierte a número
 - (c) JavaScript interpreta "6" como un literal numérico
- 74 ¿En qué orden conviene procesar un dato que llega de afuera, como un formulario o un archivo?
- (a) Convertir, usar y validar al mostrar el resultado
 - (b) Normalizar, convertir y validar antes de usarlo
 - (c) Validar el texto crudo y convertir al final
- 75 ¿Qué devuelve String(null) ?
- (a) null
 - (b) TypeError
 - (c) "null"
- 76 ¿Qué devuelve Number("") y por qué requiere cuidado en un campo obligatorio?
- (a) 0; podría aceptarse una omisión como si fuera un cero
 - (b) NaN; la omisión se detecta sin más control
 - (c) undefined; se distingue fácilmente de cero
- 77 ¿Por qué Number([]) devuelve 0?
- (a) Number devuelve la longitud del array vacío
 - (b) Se convierte primero en "" y luego en 0
 - (c) Un array vacío es falsy y false vale 0

78 Para el texto "42px", ¿qué diferencia hay entre `Number` y `parseInt`?

- ☐ a Ambos dan 42
- ☐ b `Number` da 42 y `parseInt` da NaN
- ☐ c `Number` da NaN y `parseInt` da 42

79 ¿Por qué `Boolean("false")` no sirve para interpretar esa palabra?

- ☐ a Devuelve false, pero no reconoce "FALSE"
- ☐ b Devuelve true: la cadena no está vacía
- ☐ c Lanza `TypeError` al no ser un booleano

80 Un entero muy grande llega como texto, en la variable `texto`. ¿Cómo se convierte a `bigint` sin perder dígitos?

- ☐ a `BigInt(texto)`
- ☐ b `BigInt(Number(texto))`
- ☐ c `BigInt(parseInt(texto))`

81 ¿Qué resultado produce `"8" * "3"`?

- ☐ a "83"
- ☐ b NaN
- ☐ c 24

82 ¿Qué produce `1 + 2 + "3"`?

- ☐ a "33"
- ☐ b "123"
- ☐ c 6

83 ¿Por qué `"20" < "3"` puede ser true?

- ☐ a Ambos se convierten a número antes de comparar
- ☐ b Se compara la longitud de las cadenas
- ☐ c Ambos son strings y se comparan carácter por carácter

84 ¿Qué devuelve `0 === false`?

- ☐ a true
- ☐ b false
- ☐ c undefined

85 Cuando JavaScript necesita convertir un objeto a un valor primitivo, ¿qué método usa primero si el objeto lo define?

- ☐ a `Symbol.toPrimitive`
- ☐ b `Object.prototype.valueOf`
- ☐ c `toString`

86 Un formulario acepta números con coma decimal, como "3,5". ¿Qué hace falta además de llamar a `Number`?

- ☐ a Reemplazar la coma por punto y nada más
- ☐ b Definir el formato aceptado y normalizarlo
- ☐ c Usar `parseFloat`, que ya acepta la coma

07 El tipo `symbol` y los protocolos de JavaScript

87-101

87 ¿Qué devuelve `Symbol("id") === Symbol("id")`?

- ☐ a true
- ☐ b undefined
- ☐ c false

88 En `Symbol("interno")`, ¿para qué sirve el texto "interno"?

- ☐ a Hace iguales los símbolos con ese texto
- ☐ b Convierte el símbolo en clave string
- ☐ c Ayuda a depurar; no define identidad

89 Un objeto tiene la propiedad string `id` y otra bajo `Symbol("id")`. ¿Qué relación guardan?

- ☐ a La segunda sobrescribe la primera
- ☐ b Son claves distintas
- ☐ c Ambas se enumeran con el mismo nombre

90 ¿Qué ocurre con una clave `symbol` al llamar `Object.keys` sobre un objeto?

- ☐ a No aparece en el resultado
- ☐ b Aparece convertida a string
- ☐ c Se elimina del objeto

91 ¿Qué función devuelve las claves `symbol` propias de un objeto?

- ☐ a `Object.getOwnPropertyNames`
- ☐ b `Object.entries`
- ☐ c `Object.getOwnPropertySymbols`

92 ¿Copia `{ ...obj }` las propiedades de `obj` cuya clave es un `symbol`?

- ☐ a No, se omiten las claves `symbol`
- ☐ b Sí
- ☐ c Solo si la clave se creó con `Symbol.for`

93 ¿Qué devuelve `Symbol.for("curso") === Symbol.for("curso")`?

- ☐ a true
- ☐ b false
- ☐ c `TypeError`

94 ¿Qué devuelve `Symbol.keyFor(s)` si `s` se creó con `Symbol()`, fuera del registro global?

- ☐ a Su descripción
- ☐ b undefined
- ☐ c La cadena "local"

95 ¿Qué expresión convierte un `symbol` en texto sin lanzar error?

- ☐ a `String(símbolo)`
- ☐ b `"id=" + símbolo`
- ☐ c `Number(símbolo)`

96 ¿Qué símbolo especial debe usar un objeto como nombre de un método para poder recorrerse con `for...of`?

- ☐ a `Symbol.hasInstance`
- ☐ b `Symbol.toStringTag`
- ☐ c `Symbol.iterator`

97 El método `Symbol.iterator` de un objeto devuelve un iterador. ¿Qué método debe tener ese iterador?

- ☐ a `value`, que devuelve el elemento actual
- ☐ b `next`, que devuelve `value` y `done`
- ☐ c `done`, que indica si quedan elementos

98 ¿Qué símbolo especial busca `for await...of` para recorrer valores que llegan de forma asíncronica?

- ☐ a `Symbol.isConcatSpreadable`
- ☐ b `Symbol.species`
- ☐ c `Symbol.asyncIterator`

99 ¿Qué tipo de valor debe devolver un método `Symbol.prototype.toPrimitive`?

- a Un valor primitivo
- b El propio objeto
- c Un string con la descripción

100 Si un objeto define `Symbol.toStringTag`, ¿qué cambia?

- a El nombre que muestra `typeof`
- b El texto de `Object.prototype.toString`
- c La clave con que lo guarda `JSON.stringify`

101 Un objeto tiene índices numéricos y `length`, como un array. ¿Qué símbolo hace que `concat` lo expanda elemento por elemento?

- a `Symbol.hasInstance`
- b `Symbol.keyFor`
- c `Symbol.isConcatSpreadable`

08 Arrays y matrices

102-125

102 ¿Cuándo es un array la mejor estructura para guardar una colección?

- a Cuando importan el orden y las posiciones
- b Cuando cada clave es un objeto arbitrario
- c Cuando los duplicados deben desaparecer

103 ¿Qué diferencia hay entre `Array.of(3)` y `Array(3)`?

- a `Array.of(3)` es `[3]`; `Array(3)` tiene tres huecos
- b Ambos crean un array con tres huecos vacíos
- c El primero tiene tres huecos; el segundo contiene 3

104 ¿Qué expresión comprueba de forma confiable si `x` es un array?

- a `typeof x === "array"`
- b `x instanceof Object`
- c `Array.isArray(x)`

105 Con `const a = ["rojo", "azul"]`, ¿qué devuelve `a.at(-1)`?

- a "rojo"
- b "azul"
- c `undefined`

106 ¿Qué ocurre al asignar `a.length = 1` a `a = [1, 2, 3]`?

- a Se descartan los elementos 2 y 3 y la longitud pasa a 1
- b Los elementos 2 y 3 quedan ocultos pero existen
- c Lanza `TypeError: length es de solo lectura`

107 ¿Qué devuelve `1 in [1, , 3]`?

- a `true`
- b `false`
- c `undefined`

108 ¿Qué devuelve `[10, 20, 30, 40].slice(1, 3)`?

- a `[20, 30, 40]`
- b `[10, 20, 30]`
- c `[20, 30]`

109 ¿Qué devuelve `push` al agregar un elemento a un array?

- a El elemento agregado
- b Una copia del array
- c La nueva longitud

110 ¿Qué devuelve `splice` además de modificar el array original?

- a Una copia del array original antes del cambio
- b Un array con los elementos eliminados
- c La nueva longitud del array

111 ¿Qué expresión devuelve `true` porque encuentra `NaN` dentro del array?

- a `[NaN].includes(NaN)`
- b `[NaN].indexOf(NaN) >= 0`
- c `NaN === NaN`

112 ¿Qué devuelve `find` si ningún elemento cumple la condición?

- a `undefined`
- b `-1`
- c Un array vacío

113 ¿Qué bucle recorre directamente los valores de un array?

- a `for...in`
- b `for...of`
- c `switch`

114 ¿Qué método produce un array nuevo con un resultado por elemento?

- a `forEach`
- b `pop`
- c `map`

115 `filter` devuelve un array nuevo con algunos objetos del array original. ¿Qué relación tienen esos objetos con los del original?

- a Son los mismos objetos: `filter` copia solo las referencias
- b Cada objeto se copia en forma superficial
- c Se copian solo los objetos modificados

116 ¿Por qué conviene pasar un valor inicial a `reduce`?

- a Hace que `reduce` recorra el array de derecha a izquierda
- b Fija el acumulador y cubre el array vacío
- c Evita que el callback reciba el índice

117 ¿Qué devuelve `forEach`?

- a Un array transformado
- b El último elemento
- c `undefined`

118 ¿Qué resultado produce el siguiente código?

```
const datos = [1, [2, [3]]];
datos.flat(2);
```

- a Un array plano con 1, 2 y 3
- b Un array con el 3 todavía anidado
- c Un array con el 1 y el 2 anidados

119 ¿Por qué `[2, 10, 1].sort()` no da necesariamente el orden numérico esperado?

- a `sort` sin comparador ordena de mayor a menor
- b `sort` ordena según el orden de inserción original
- c Sin comparador, `sort` convierte a string y compara texto

120 Después de `const copia = [...original]`, donde `original` contiene objetos, ¿qué comparten ambos arrays?

- a El mismo array exterior, con otro nombre
- b Las referencias a los objetos internos
- c Nada: los objetos se copian también

- 121 ¿Qué problema tiene crear una matriz de 3×3 con `Array(3).fill(Array(3).fill(0))`?
- (a) Las tres filas son el mismo array, así que cambiar una cambia todas
 - (b) Las filas quedan con huecos en lugar de ceros
 - (c) Solo la primera fila queda llena de ceros
- 122 Para un array con los valores 10, 20, 30, 40 y 50, ¿qué devuelve `slice(-2)`?
- (a) El array original reducido a 40 y 50
 - (b) Un array nuevo con 40 y 50
 - (c) Un array nuevo con 10 y 20
- 123 Para los valores 1, 2 y 3, ¿qué devuelven respectivamente `some(n => n > 2)` y `every(n => n > 2)`?
- (a) true y true
 - (b) false y false
 - (c) true y false
- 124 Tras ejecutar este código, ¿qué ocurre con `original` y `copia`?
- ```
const original = [3, 1, 2];
const copia = original.toSorted((a, b) => a - b);
```
- (a) Ambos quedan ordenados de menor a mayor
  - (b) `original` no cambia; `copia` queda ordenada
  - (c) `original` queda ordenado; `copia` no cambia
- 125 ¿Por qué `Array.from({ length: 3 }, () => Array(3).fill(0))` evita el problema de las filas compartidas?
- (a) Todas las filas usan la misma referencia a un único array
  - (b) Convierte la matriz en un objeto sin índices
  - (c) La función crea un array nuevo para cada fila

## 09 Objetos, propiedades y referencias

126–147

- 126 ¿Qué estructura conviene para representar un alumno con campos fijos, como nombre y legajo?
- (a) Un objeto
  - (b) Un Set
  - (c) Una cadena separada por comas
- 127 Si un objeto se crea con `{ 10: "diez" }`, ¿cómo aparece esa clave en `Object.keys`?
- (a) Como el string "10"
  - (b) Como el número 10
  - (c) Como un symbol
- 128 Con `const campo = "precio"`, ¿qué expresión lee la propiedad `precio` de `producto` usando la variable?
- (a) `producto.campo`
  - (b) `producto["campo"]`
  - (c) `producto[campo]`
- 129 ¿Qué efecto tiene `delete producto.stock`?
- (a) La conserva con valor undefined
  - (b) Quita la propiedad stock
  - (c) La convierte en null

- 130 ¿Qué diferencia hay entre `Object.hasOwn(obj, "x")` y `"x" in obj`?
- (a) `in` considera propiedades heredadas; `hasOwn` no
  - (b) `hasOwn` considera heredadas; `in` no
  - (c) Ambas revisan también las propiedades heredadas
- 131 `alumno.direccion` puede ser `null` o `undefined`. ¿Qué evita escribir `alumno.direccion?.ciudad`?
- (a) Validar cualquier campo obligatorio
  - (b) Leer ciudad cuando `direccion` es `null` o `undefined`
  - (c) Que ciudad valga `undefined` si falta
- 132 Con `const nombre = "Ana"`, ¿qué crea `{ nombre }`?
- (a) Un Set que contiene el string "Ana"
  - (b) Un bloque de código que evalúa `nombre`
  - (c) La propiedad `nombre` con valor "Ana"
- 133 ¿Por qué no conviene definir con una función flecha un método de objeto que usa `this`?
- (a) No puede devolver valores
  - (b) No tiene `this` propio
  - (c) Su `this` es el objeto donde se define
- 134 En un literal de objeto, ¿qué sintaxis usa el contenido de la variable `campo` como nombre de la clave?
- (a) `{ campo: valor }`
  - (b) `{ "campo": valor }`
  - (c) `{ [campo]: valor }`
- 135 ¿Qué función devuelve pares [clave, valor] de propiedades propias enumerables?
- (a) `Object.entries`
  - (b) `Object.getOwnPropertyNames`
  - (c) `Object.hasOwn`
- 136 ¿Qué devuelve `({ x: 1 }) === ({ x: 1 })`?
- (a) true, porque sus campos coinciden
  - (b) undefined, porque no se comparan objetos
  - (c) false, porque son objetos distintos
- 137 `a` es un objeto. Después de `const b = a; b.nombre = "Ana";`, ¿qué valor tiene `a.nombre`?
- (a) El valor anterior, porque `b` es una copia
  - (b) El nuevo valor, porque `a` y `b` son el mismo objeto
  - (c) undefined, porque `b` tomó la propiedad
- 138 `original` tiene una propiedad `usuario` que es un objeto. Después de `const copia = { ...original }`, ¿qué comparación da true?
- (a) `copia.usuario === original.usuario`
  - (b) `copia === original`
  - (c) `copia.usuario === structuredClone(original.usuario)`
- 139 `base` tiene la propiedad `tema: "claro"`. ¿Qué valor de `tema` tiene el objeto `{ ...base, tema: "oscuro" }`?
- (a) "claro"
  - (b) "oscuro"
  - (c) undefined

- 140** ¿Qué argumento modifica `Object.assign(destino, origen)` ?
- (a) origen
  - (b) Ninguno: devuelve un objeto nuevo
  - (c) destino
- 
- 141** Al copiar un objeto que contiene otros objetos, ¿qué hace `structuredClone` que el spread `{ ...obj }` no hace?
- (a) Puede crear objetos internos nuevos en lugar de compartirlos
  - (b) Copia también las funciones de los objetos
  - (c) Solo copia el primer nivel del objeto
- 
- 142** Con `const { nombre: nombreCompleto } = persona`, ¿qué variable local se crea?
- (a) nombre
  - (b) nombreCompleto
  - (c) persona.nombreCompleto
- 
- 143** ¿Por qué `JSON.parse(JSON.stringify(obj))` no sirve para clonar cualquier objeto?
- (a) Copia solo el primer nivel del objeto
  - (b) Modifica el objeto original al serializarlo
  - (c) Pierde undefined, funciones y fechas, y falla con ciclos
- 
- 144** ¿Qué añade `Reflect.ownKeys(obj)` frente a `Object.keys(obj)` ?
- (a) También claves symbol y propiedades no enumerables
  - (b) Las claves heredadas del prototipo
  - (c) Los valores junto con cada clave
- 
- 145** Si se aplica `Object.freeze` a un objeto que contiene otro objeto, ¿qué cambio puede seguir siendo posible?
- (a) Modificar el objeto interior, si no se congeló
  - (b) Reasignar una propiedad directa con otro objeto
  - (c) Agregar propiedades directas si no existían antes
- 
- 146** En `const { idioma = "es" } = configuracion`, ¿cuándo se usa "es" ?
- (a) Cuando idioma vale null o undefined
  - (b) Cuando idioma es una cadena vacía
  - (c) Cuando la propiedad falta o su valor es undefined
- 
- 147** ¿Qué hace `Object.fromEntries` con una colección de pares clave y valor?
- (a) Devuelve un Map con esos pares
  - (b) Construye un objeto con esos pares como propiedades
  - (c) Devuelve un array de claves

## 10 Map, Set y colecciones especializadas 148–162

- 148** ¿Cuándo conviene usar un Map en lugar de un objeto común?
- (a) Cuando se necesita convertir a JSON directo
  - (b) Cuando importa el orden por posición numérica
  - (c) Cuando las claves salen de los datos y pueden ser de cualquier tipo
- 
- 149** ¿Puede un objeto usarse como clave de Map sin convertirse en string?
- (a) No, Map convierte la clave en string
  - (b) Sí
  - (c) Solo si tiene una propiedad id

- 150** Un Map guarda el valor `undefined` en una clave. ¿Cómo se distingue esa clave de una que no existe?
- (a) Con `mapa.has(clave)`
  - (b) Con `mapa.get(clave) === undefined`
  - (c) Comparando el tamaño con cero

- 151** ¿En qué orden recorre pares un Map?
- (a) En orden alfabético de claves
  - (b) En orden de inserción
  - (c) En orden numérico de valores

- 152** ¿En qué orden recibe argumentos el callback de `Map.forEach`?
- (a) clave, valor, mapa
  - (b) índice, valor, mapa
  - (c) valor, clave, mapa

- 153** Hay que buscar muchos alumnos por legajo dentro de un array. ¿Qué se gana al armar antes un Map con el legajo como clave?
- (a) Busca por legajo directamente, sin recorrer el array cada vez
  - (b) Ordena los alumnos por número de legajo
  - (c) Evita que haya dos alumnos con el mismo nombre

- 154** ¿Qué tamaño tiene `new Set(["js", "node", "js"])` ?
- (a) 3
  - (b) 1
  - (c) 2

- 155** ¿Qué valor tiene `new Set([ { id: 1 }, { id: 1 } ]).size` ?
- (a) 1
  - (b) 2
  - (c) 0

- 156** Si se agrega a un Set un valor que ya contiene, ¿qué pasa con ese valor?
- (a) Queda en su lugar original
  - (b) Se mueve al final del Set
  - (c) Se agrega otra copia al final

- 157** ¿Qué contiene la intersección de dos conjuntos?
- (a) Los valores de uno u otro conjunto
  - (b) Solo valores presentes en ambos
  - (c) Valores presentes en exactamente uno

- 158** ¿Qué estructura conviene si importan el orden de los elementos y también sus repeticiones?
- (a) Set
  - (b) WeakSet
  - (c) Array

- 159** Si los valores de un Map son objetos, ¿qué comparte `new Map(original)` con el Map original?
- (a) Las referencias a esos objetos
  - (b) El mismo Map, con otro nombre
  - (c) Nada: los valores se clonan también

- 160** ¿Qué diferencia principal tiene un WeakMap respecto de un Map?
- (a) No impide que el recolector libere sus claves objeto
  - (b) Permite iterar todas sus claves
  - (c) Acepta claves primitivas y objetos

161 ¿Para qué sirve WeakSet?

- a Guardar valores primitivos sin duplicados
- b Marcar objetos sin impedir su liberación
- c Listar objetos en orden de inserción

162 ¿Qué método de Set indica si un valor pertenece al conjunto?

- a set.indexOf(valor)
- b set.get(valor)
- c set.has(valor)

## 11 Estructuras de control

163–182

163 ¿Qué hace más legible la condición de un `if` que expresa una regla importante?

- a Un valor truthy sin comparar
- b Una condición con nombre que explique la regla
- c Una asignación dentro del `if`

164 Cada vez que se ejecuta `if (condición) { A } else { B }`, ¿qué está garantizado?

- a Se ejecuta uno de los dos caminos
- b Se ejecutan A y B en orden
- c No se ejecuta ninguno aunque la condición sea válida

165 Una cadena de `if...else if` clasifica notas en dos grupos: 8 o más, y 6 o más. ¿Qué condición debe ir primero?

- a La de 6 o más
- b Cualquiera, pues se evalúan ambas ramas
- c La de 8 o más

166 Una función empieza con `if (!pedido) return;`. ¿Qué se logra con esa guarda?

- a Hace que todas las condiciones sean globales
- b Obliga a usar `switch`
- c Deja el camino válido con menos anidamiento

167 ¿Cuándo conviene anidar dos `if` en lugar de unir las condiciones con `&&`?

- a Cuando una decisión solo tiene sentido dentro de la otra
- b Cuando ambas condiciones son independientes y triviales
- c Cuando hay que garantizar que se evalúen las dos condiciones

168 ¿Qué equivale a `!(tieneSaldo && hayStock)`?

- a `!tieneSaldo && !hayStock`
- b `!tieneSaldo || !hayStock`
- c `tieneSaldo || hayStock`

169 ¿Cómo compara `switch` el valor con cada `case`?

- a Con igualdad flexible (`==`)
- b Con igualdad estricta
- c Con comparación estructural profunda

170 ¿En qué situación conviene usar `switch`?

- a Clasificar intervalos numéricos solapados
- b Repetir hasta que cambie una condición
- c Elegir según códigos fijos de un estado

171 ¿Qué pasa en `switch` si un `case` no termina con `break`, `return` o `throw`?

- a Sigue ejecutando el `case` siguiente hasta encontrar un corte
- b Sale del `switch` al final del `case`
- c Se produce `SyntaxError` al analizar el código

172 ¿Para qué sirve encerrar entre llaves el código de cada `case` de un `switch`?

- a Para que cada `case` tenga su propio alcance
- b Para que el `break` sea opcional en ese `case`
- c Para que `case` compare con igualdad flexible

173 `intentos` empieza en 0. ¿Qué le falta a `while (intentos < 3) { ejecutar(); }` para terminar?

- a Un `else` que termine el bucle
- b Incrementar `intentos` en el cuerpo
- c Declarar `intentos` con `const`

174 ¿Qué bucle ejecuta el cuerpo al menos una vez antes de evaluar su condición?

- a `while`
- b `for` clásico
- c `do..while`

175 ¿En qué orden se ejecutan las partes de un `for` clásico?

- a Inicio, condición, cuerpo, actualización, condición
- b Inicio, condición, actualización, cuerpo, condición
- c Inicio, cuerpo, condición, actualización, cuerpo

176 ¿Qué bucle recorre valores de un array o Set?

- a `for...in`
- b `switch`
- c `for...of`

177 ¿Qué recorre `for...in` sobre un objeto?

- a Los valores de las propiedades propias
- b Claves enumerables, también las heredadas
- c Las claves propias, incluidas las `symbol`

178 ¿Qué hace `break` dentro de un bucle?

- a Salta solo a la vuelta siguiente
- b Reinicia el contador
- c Termina el bucle más cercano

179 ¿Qué hace `continue` dentro de un bucle?

- a Pasa a la vuelta siguiente
- b Termina el bucle más cercano
- c Sale de todos los bucles anidados

180 ¿Qué permite `break etiqueta` en bucles anidados?

- a Salir del bucle interior, como un `break` común
- b Salir directamente del bucle que tiene esa etiqueta
- c Volver al comienzo del bucle exterior

181 Un bucle recorre un array por índice y elimina elementos con `splice`. ¿Por qué puede saltarse elementos?

- a El siguiente se corre al índice actual
- b `splice` deja un hueco en esa posición
- c `splice` reordena el resto del array

- 182** ¿Cuándo conviene usar el operador ternario `condición ? a : b` ?
- (a) Al encadenar varias decisiones anidadas
  - (b) Al ejecutar una acción sin valor de retorno
  - (c) Al elegir un valor entre dos alternativas de forma breve

## 12 Errores y excepciones

183–200

- 183** Dentro de un bloque, `JSON.parse` lanza un error porque el texto no es JSON válido. ¿Qué pasa con la sentencia siguiente de ese bloque?
- (a) Se ejecuta con el valor `undefined`
  - (b) No se ejecuta: la excepción corta el bloque y se propaga
  - (c) Se ejecuta y el error se registra en consola
- 184** Si una sentencia dentro de `try` lanza un error, ¿dónde continúa la ejecución?
- (a) Al comienzo del `try`
  - (b) Directo al `finally`
  - (c) Al `catch` correspondiente
- 185** ¿Por qué se recomienda lanzar `new Error(mensaje)` y no una cadena?
- (a) Permite capturarlo con `catch`
  - (b) Incluye nombre, mensaje y pila
  - (c) Hace que el error no detenga el programa
- 186** Una función `dividir` no admite divisor cero y, si lo recibe, hace `throw new RangeError(...)`. ¿Qué expresa ese error?
- (a) Que la entrada incumple el contrato de esa función
  - (b) Que JavaScript lanza al dividir un number por cero
  - (c) Que cero se convierte en `null`
- 187** Una función busca un alumno, y que no exista es un resultado habitual. ¿Qué conviene que devuelva si no lo encuentra?
- (a) Lanzar una excepción al no encontrarlo
  - (b) Un objeto alumno con campos vacíos
  - (c) Un resultado documentado como `null`
- 188** ¿Qué error lanza JavaScript cuando una operación recibe un valor de un tipo que no corresponde?
- (a) `TypeError`
  - (b) `RangeError`
  - (c) `URIError`
- 189** ¿Qué error lanza `JSON.parse` si el texto está mal formado?
- (a) `ReferenceError`
  - (b) `SyntaxError`
  - (c) `AggregateError`
- 190** Una función de stock lanza un error que incluye las propiedades `disponible` y `requerido`. ¿Qué ventaja tiene eso?
- (a) Hace innecesario el mensaje de error
  - (b) Permite decidir con datos, no con el texto
  - (c) Hace que el error no necesite capturarse
- 191** Se captura un error técnico y se lanza otro, más comprensible para la aplicación. ¿Qué propiedad del nuevo error guarda el error original?
- (a) `stackLength`
  - (b) `responseText`
  - (c) `cause`

- 192** ¿Por qué conviene evitar un único `try` que envuelva el parseo, la validación, el guardado y la notificación?
- (a) Se vuelve difícil saber qué etapa falló y cómo recuperarse
  - (b) Impide que `finally` se ejecute
  - (c) Hace más lento el código del `try`

- 193** Un `catch` recibe un error que no sabe manejar. ¿Cómo conviene relanzarlo para conservar su tipo y su pila?
- (a) `throw new Error(error.message)` sin causa
  - (b) `throw error`
  - (c) `return false`

- 194** ¿Cuándo se ejecuta el bloque `finally` ?
- (a) Siempre: tras un éxito, un `return` o una excepción del `try`
  - (b) Solo cuando el `try` lanza una excepción
  - (c) Solo si el `try` no terminó con `return`

- 195** ¿Qué riesgo tiene escribir `return` dentro de un bloque `finally` ?
- (a) Impide que `finally` termine de limpiar
  - (b) Relanza la excepción que se había capturado
  - (c) Puede tapar el `return` o el error del `try`

- 196** Dentro de un `try` se llama a una función que devuelve una promesa, sin `await`. Si la promesa se rechaza después, ¿por qué el `catch` no lo captura?
- (a) `catch` no puede recibir objetos `Promise`
  - (b) El `try` ya terminó cuando llega el rechazo
  - (c) El rechazo se convierte en `undefined` sin error

- 197** Después de `const response = await fetch(url)`, ¿qué hay que comprobar para detectar una respuesta 404?
- (a) `response.ok` o `response.status`
  - (b) Si la promesa de `fetch` fue rechazada
  - (c) Si `response.json()` devuelve `null`

- 198** ¿Qué devuelve `Promise.allSettled` cuando algunas tareas fallan?
- (a) Solo los valores de las tareas exitosas
  - (b) El primer rechazo, como `Promise.all`
  - (c) Un objeto por tarea con su estado: `fulfilled` o `rejected`

- 199** Si la aplicación falla por un error interno, ¿qué conviene mostrarle a quien la usa?
- (a) El mensaje técnico original del error
  - (b) Un mensaje claro, sin detalles internos
  - (c) El código de error y la pila resumida

- 200** En una tienda, quedarse sin stock es un caso frecuente. ¿Cómo conviene que la función que procesa la compra informe ese caso?
- (a) Lanzar un `Error` y capturarlo arriba
  - (b) Devolver `undefined` sin explicación
  - (c) Un resultado con `ok` `false` y motivo

## 13 Funciones

201–226

- 201** ¿Qué diferencia hay entre `sumar` y `sumar(2, 3)` ?
- (a) Uno nombra la función; el otro la ejecuta con 2 y 3
  - (b) Ambos llaman; el primero sin argumentos
  - (c) El primero es un `string` con el nombre

**202** En `function presentar(nombre) {}`, ¿qué es `nombre`?

- ☐ a Un argumento de la llamada
- ☐ b Un parámetro de la función
- ☐ c Una variable global implícita

**203** Una función se llama sin pasarle un argumento, y ese parámetro no tiene valor predeterminado. ¿Qué valor tiene el parámetro?

- ☐ a undefined
- ☐ b null
- ☐ c ReferenceError

**204** ¿Se puede llamar a `duplicar()` en una línea anterior a su declaración `function duplicar(n) {...}`, dentro del mismo alcance?

- ☐ a No, lanza ReferenceError
- ☐ b Solo si está en modo estricto
- ☐ c Sí

**205** ¿Se puede llamar a una función guardada con `const f = () => ...` en una línea anterior a esa declaración?

- ☐ a Sí, se eleva igual que una declaración completa
- ☐ b No, sigue las reglas de const
- ☐ c Solo si no recibe argumentos

**206** Una función flecha sin llaves debe devolver un objeto literal. ¿Con qué hay que rodear el objeto?

- ☐ a Corchetes de array
- ☐ b Comillas
- ☐ c Paréntesis

**207** Dentro de un método común se define una función flecha. ¿Qué `this` usa la flecha?

- ☐ a El mismo this que tiene el método que la contiene
- ☐ b Uno nuevo creado por la flecha
- ☐ c globalThis, en modo no estricto

**208** Un método usa `this`. ¿Qué puede pasar si se guarda ese método en una variable y se lo llama desde ella?

- ☐ a El objeto se clona
- ☐ b El método se convierte en generador
- ☐ c Puede perder el receptor original

**209** ¿Qué hace `persona.saludar.bind(persona)`?

- ☐ a Llama a saludar con `this = persona`
- ☐ b Crea una función nueva con `this` fijado en `persona`
- ☐ c Cambia saludar en el propio objeto

**210** Con `function f(x = 3) { return x; }`, ¿con qué argumento `f` devuelve 3?

- ☐ a undefined
- ☐ b null
- ☐ c "" (cadena vacía)

**211** ¿Dónde debe ubicarse `...resto` en la lista de parámetros?

- ☐ a Al comienzo
- ☐ b Al final
- ☐ c En cualquier posición

**212** ¿Qué ventaja tiene el parámetro `rest (...args)` frente al objeto `arguments`?

- ☐ a Un `this` propio dentro de las flechas
- ☐ b La cantidad de argumentos recibidos
- ☐ c Un array real de argumentos, también en flechas

**213** La función `function f({ a, b }) {...}` se llama como `f(null)`. ¿Qué ocurre?

- ☐ a Lanza `TypeError` al intentar desestructurar `null`
- ☐ b Usa los valores predeterminados
- ☐ c Desestructura un objeto vacío

**214** Una función recibe un número y dentro le asigna el doble a su parámetro. ¿Cambia la variable que se pasó como argumento?

- ☐ a Sí, porque se pasa por referencia
- ☐ b No
- ☐ c Solo si la función usa `return`

**215** Una función recibe un objeto y modifica una de sus propiedades. ¿Qué ve después el código que la llamó?

- ☐ a El cambio, porque ambos apuntan al mismo objeto
- ☐ b El objeto sin cambios, porque se copió
- ☐ c El cambio, solo si la función lo devuelve

**216** Una función recibe un objeto y asigna un objeto nuevo a su parámetro. ¿A qué apunta la variable que se pasó como argumento?

- ☐ a Pasa a apuntar al nuevo objeto
- ☐ b Se vuelve undefined
- ☐ c Permanece apuntando al objeto original

**217** ¿Qué devuelve una función que solo hace `console.log` y no retorna?

- ☐ a El texto impreso
- ☐ b La última variable local
- ☐ c undefined

**218** ¿Qué pasa si se escribe `return` y el objeto literal que se quiere devolver empieza en la línea siguiente?

- ☐ a Que el objeto se devuelva normalmente
- ☐ b Que la función devuelva undefined
- ☐ c Que se produzca un `SyntaxError`

**219** Una función necesita devolver varios datos. ¿Qué estructura les da nombres claros?

- ☐ a Un objeto con una propiedad por cada dato
- ☐ b Varios `return`, uno por cada dato
- ☐ c Un array con los datos en orden

**220** Una función interior usa una variable exterior, y esa variable cambia después de crear la función. ¿Qué valor ve la función interior?

- ☐ a Una copia del valor inicial
- ☐ b El valor que tenía al declarar la clausura
- ☐ c La variable, con su valor actual

**221** ¿Qué devuelve siempre una función marcada `async`?

- ☐ a Un valor síncrono sin envoltorio
- ☐ b Una promesa
- ☐ c Un generador

**222** ¿Cuándo se ejecuta el código dentro de una función generadora?

- a** Cuando se llama a next()
- b** Al llamar a la función generadora
- c** Al declararse la función

**223** Una función usa la variable `tasa` sin declararla adentro. ¿Dónde la busca JavaScript?

- a** En el objeto desde el que la llamaron
- b** En el alcance del lugar donde se la llama
- c** En los alcances del lugar donde fue definida la función

**224** `crearContador()` devuelve funciones que modifican una variable local. Si se llama dos veces a `crearContador()`, ¿qué comparten los dos contadores?

- a** Nada: cada llamada tiene su propio estado
- b** El mismo contador, porque la función es única
- c** Solo el valor inicial del contador

**225** ¿Qué muestra `funciones.map(fn => fn())` al final de este código?

```
const funciones = [];
for (let i = 0; i < 3; i += 1) {
 funciones.push(() => i);
}
```

- a** Un array con 3, 3 y 3
- b** Un array con 0, 1 y 2
- c** Un array con 2, 2 y 2

**226** Al documentar una función que recibe un callback, ¿qué conviene aclarar?

- a** Si lo ejecuta enseguida, varias veces o más tarde
- b** Que lo ejecuta exactamente una vez y de forma inmediata
- c** Que el callback no puede devolver un valor

## 14 Programación funcional y pipelines de datos 227-246

**227** ¿Qué caracteriza a una función de orden superior?

- a** Se llama a sí misma
- b** Recibe o devuelve funciones
- c** No tiene efectos secundarios

**228** ¿Qué condiciones cumple una función pura?

- a** Mismo resultado, aunque modifique variables externas
- b** Ningún efecto externo, aunque cambie el resultado
- c** Siempre el mismo resultado y ningún efecto externo observable

**229** Una función que calcula un impuesto lee la variable global `tasaActual`. ¿Por qué eso la vuelve impura?

- a** Leer una variable global la modifica
- b** Toda multiplicación con decimales es impura
- c** Su resultado depende de algo externo

**230** En un programa hecho con funciones puras, ¿dónde conviene ubicar la lectura de archivos y la escritura de resultados?

- a** Dentro de cada transformación del pipeline
- b** En los bordes del programa: al entrar y al salir
- c** En una función auxiliar por cada dato

**231** ¿En qué se diferencia `{ ...alumno, nota: 8 }` de hacer `alumno.nota = 8`?

- a** Crea una versión exterior nueva con nota 8
- b** Clona profundamente cualquier objeto anidado
- c** Borra todas las otras propiedades

**232** Una función crea un Map dentro de su cuerpo, lo modifica y no toca sus argumentos. ¿Puede ser pura?

- a** No, cualquier mutación la vuelve impura
- b** Solo si el Map se congela al terminar
- c** Sí, si no hay efectos visibles afuera

**233** ¿Cuántos elementos agrega `map` al resultado por cada elemento del array?

- a** Cero o una, según el callback
- b** Una
- c** Tantas como devuelva el callback

**234** ¿Qué decide la función que se le pasa a `filter`?

- a** Si se conserva cada elemento
- b** El orden de inserción global
- c** La cantidad exacta de llamadas futuras

**235** En `reduce`, ¿qué papel cumple el valor inicial?

- a** Es el primer elemento que se procesa
- b** Es el acumulador de la primera vuelta
- c** Es el valor que se devuelve si falla

**236** ¿Qué método de array deja de recorrer apenas encuentra un elemento que cumple la condición y devuelve un booleano?

- a** every
- b** forEach
- c** some

**237** ¿Cuándo es `forEach` el método adecuado?

- a** Cuando solo importa el efecto por elemento
- b** Cuando se necesita un array transformado
- c** Cuando hay que cortar el recorrido antes

**238** ¿Cuántos elementos puede aportar al resultado cada llamada de la función que recibe `flatMap`?

- a** Cero, uno o varios elementos
- b** Exactamente un elemento, como map
- c** Un array anidado sin aplanar

**239** ¿Qué método ordena un array devolviendo un array nuevo, sin modificar el original?

- a** sort
- b** toReversed
- c** toSorted

**240** ¿Qué hacen, en orden, los pasos `filter`, `map` y `reduce` encadenados?

- a** Acumular, ordenar y borrar
- b** Seleccionar, transformar y acumular
- c** Crear un objeto, clonarlo y serializarlo

- 241** ¿Qué hace una función `pipe` que recibe varias funciones y las aplica de izquierda a derecha?
- ☐ a Ejecuta todas las funciones sobre la entrada original sin relación
  - ☐ b Vuelve cada función recursiva
  - ☐ c Pasa la salida de una transformación a la siguiente
- 
- 242** Si `esImpar = n => !esPar(n)`, ¿qué papel tiene `esPar`?
- ☐ a Un predicado que se reutiliza
  - ☐ b Una función de orden superior
  - ☐ c Un callback que se ejecuta después
- 
- 243** ¿Qué ocurre al ejecutar `array.filter(...).map(...)`?
- ☐ a Nada se ejecuta hasta pedir el resultado
  - ☐ b Cada uno crea un array intermedio
  - ☐ c Se combinan en un único recorrido
- 
- 244** ¿Qué ventaja tiene usar un generador para producir una secuencia muy grande?
- ☐ a Calcular todos los valores por adelantado
  - ☐ b Ejecutarse en paralelo con el resto
  - ☐ c Producir solo los valores consumidos
- 
- 245** ¿Por qué `["10", "10", "10"].map(parseInt)` puede dar resultados inesperados?
- ☐ a `map` pasa el índice y `parseInt` lo usa como base
  - ☐ b `map` convierte cada string en número antes
  - ☐ c `parseInt` necesita que la base sea un string
- 
- 246** ¿Qué devuelve la función `n => { n * 2; }` al llamarla?
- ☐ a El doble de `n`
  - ☐ b `undefined`
  - ☐ c Un objeto con el doble de `n`

## 15 Recursividad y árboles binarios

247–268

- 247** ¿Qué crea cada llamada recursiva para guardar sus parámetros y variables locales?
- ☐ a Una variable global nueva
  - ☐ b Un marco propio en la pila
  - ☐ c Un hilo de ejecución propio
- 
- 248** ¿Qué hace el caso base de una función recursiva?
- ☐ a Resuelve una entrada sin hacer otra llamada
  - ☐ b Duplica la entrada para la llamada siguiente
  - ☐ c Ordena los nodos del árbol
- 
- 249** Cuando una función recursiva no está en el caso base y se llama a sí misma, ¿qué debe garantizar esa llamada?
- ☐ a Reducir el problema a la mitad
  - ☐ b Hacer al menos dos llamadas
  - ☐ c Acercarse al caso base con una entrada más pequeña
- 
- 250** En `factorial(n) = n * factorial(n - 1)`, ¿qué operación combina el resultado de la llamada recursiva?
- ☐ a La multiplicación por `n`
  - ☐ b La lectura de `n` sin cambio
  - ☐ c La creación de un array

- 251** Para sumar un array de millones de elementos, ¿por qué conviene un bucle en lugar de recursión?
- ☐ a Permite sumar sin variable acumuladora
  - ☐ b Evita un marco de pila por elemento
  - ☐ c Recorre el array en orden inverso
- 
- 252** Un documento tiene secciones que pueden contener otras secciones, sin límite de profundidad. ¿Por qué una función recursiva es adecuada para recorrerlo?
- ☐ a Las secciones forman una lista plana
  - ☐ b La profundidad se conoce de antemano
  - ☐ c Cada sección tiene la misma forma que la sección que la contiene
- 
- 253** En un árbol binario, ¿qué indica que un nodo tenga `null` como hijo?
- ☐ a Un subárbol vacío
  - ☐ b Un nodo con valor cero
  - ☐ c Una referencia al padre
- 
- 254** En un árbol binario de búsqueda, ¿dónde van los valores menores que el nodo?
- ☐ a En la rama que tenga menos nodos
  - ☐ b En su subárbol menor
  - ☐ c En el subárbol mayor, si hay lugar
- 
- 255** ¿Por qué hay que decidir una regla para los valores repetidos en un árbol binario de búsqueda?
- ☐ a Porque el árbol no admite duplicados
  - ☐ b Para que el recorrido inorden sea más rápido
  - ☐ c Para que todas las operaciones coincidan
- 
- 256** ¿En qué orden visita los nodos un recorrido inorden de un árbol binario?
- ☐ a Nodo, menor, mayor
  - ☐ b Menor, mayor, nodo
  - ☐ c Menor, nodo, mayor
- 
- 257** ¿Qué visita primero un recorrido preorden de un árbol binario?
- ☐ a El nodo antes de sus hijos
  - ☐ b La hoja más profunda
  - ☐ c El subárbol mayor antes de la raíz
- 
- 258** ¿Cuándo procesa cada nodo un recorrido postorden de un árbol binario?
- ☐ a Antes de ambos hijos
  - ☐ b Después de ambos hijos
  - ☐ c Entre el hijo menor y el mayor
- 
- 259** ¿Qué ventaja tiene recorrer un árbol con un generador?
- ☐ a Evita toda recursión en el recorrido
  - ☐ b Entrega valores a medida que se piden
  - ☐ c Recorre el árbol en varios hilos a la vez
- 
- 260** Al buscar un valor en un árbol binario de búsqueda, después de compararlo con un nodo, ¿por cuántas ramas se sigue buscando?
- ☐ a Una rama como máximo
  - ☐ b Ambas ramas
  - ☐ c Ninguna aunque el valor difiera

- 261** En una inserción recursiva en un árbol se escribe `nodo.menor = insertar(nodo.menor, valor)`. ¿Por qué se reasigna `nodo.menor`?
- (a) Para que la raíz pase a ser el nodo menor
  - (b) Para borrar el subárbol menor anterior
  - (c) insertar puede devolver un nodo nuevo donde antes había null
- 
- 262** Al insertar un valor en un árbol sin modificar el original (inserción inmutable), ¿qué nodos comparte el árbol nuevo con el anterior?
- (a) Los del camino de la raíz al nuevo nodo
  - (b) Los de ramas que no cambiaron
  - (c) Ninguno, se clona todo
- 
- 263** Si los nodos no guardan ningún contador, ¿qué hace falta para contar los nodos de un árbol?
- (a) Visitar todos los nodos
  - (b) Consultar solo la raíz
  - (c) Consultar solo el mínimo
- 
- 264** ¿Cómo se encuentra el valor mínimo en un árbol binario de búsqueda?
- (a) Bajando por la rama mayor
  - (b) Comparando todas las hojas
  - (c) Bajando por la rama menor
- 
- 265** Al eliminar una hoja de un árbol binario, ¿qué valor ocupa su lugar en el nodo padre?
- (a) La raíz original
  - (b) Una copia de ambos hijos
  - (c) null
- 
- 266** En un árbol binario de búsqueda, ¿cómo puede eliminarse un nodo con dos hijos?
- (a) Reemplazarlo por su sucesor y quitar el sucesor de su rama
  - (b) Subir su hijo mayor a su lugar
  - (c) Reemplazarlo por la raíz del árbol
- 
- 267** Para verificar que un árbol es de búsqueda, ¿por qué no alcanza con comparar cada nodo con sus hijos directos?
- (a) Los hijos pueden ser null
  - (b) Un ancestro también impone límites
  - (c) El orden también depende de la altura
- 
- 268** ¿Qué pasa si se insertan valores ya ordenados en un árbol binario de búsqueda que no se balancea?
- (a) La altura queda logarítmica
  - (b) Se forma una cadena y buscar puede requerir n comparaciones
  - (c) El árbol se balancea al insertar

## 16 Programación Asíncrona

269–281

- 269** ¿Qué devuelve siempre una función declarada `async`, incluso si retorna un valor simple?
- (a) Una promesa
  - (b) Un valor síncrono sin envoltorio
  - (c) Un iterador

**270** Una función `async` ejecuta `await` sobre una promesa que todavía no se resolvió. ¿Qué se detiene?

- (a) Todo el hilo de JavaScript
- (b) La función que llamó a la función `async`
- (c) El resto de esa función `async`

**271** ¿Por qué se usa `await` dos veces en `const respuesta = await fetch(url); const datos = await respuesta.json();`?

- (a) El segundo vuelve a pedir el recurso
- (b) Cada operación devuelve su propia promesa
- (c) `json()` necesita `await` por ser un método

**272** ¿Qué hay que hacer para que una respuesta 404 de `fetch` se trate como un error?

- (a) Revisar `respuesta.ok` o el status y lanzar un error
- (b) Esperar que `fetch` rechace la promesa
- (c) Envolver el `fetch` en `try/catch`

**273** Un botón se deshabilita mientras dura una consulta. ¿En qué bloque conviene volver a habilitarlo, tanto si la consulta sale bien como si falla?

- (a) `catch` solamente
- (b) El primer `await`
- (c) `finally`

**274** ¿Cómo se ejecutan al mismo tiempo varias tareas asíncronas independientes, esperando a que terminen todas?

- (a) `await` en cada una, una tras otra
- (b) Un `forEach async` sin esperar
- (c) `Promise.all` sobre sus promesas

**275** ¿Por qué `array.forEach(async x => { await ... })` no espera a que terminen las operaciones?

- (a) `forEach` ignora las promesas que recibe
- (b) `forEach` no acepta funciones `async`
- (c) `await` dentro de `forEach` bloquea todo el bucle

**276** Dos tareas independientes pueden fallar y se necesita conocer el resultado de cada una. ¿Qué conviene usar?

- (a) `Promise.all` y revisar cada posición
- (b) `Promise.allSettled` y revisar cada estado
- (c) `Promise.race` y esperar la primera

**277** ¿Qué ocurre si una de las promesas entregadas a `Promise.all` se rechaza?

- (a) La promesa conjunta resuelve con `undefined` en esa posición
- (b) La promesa conjunta se rechaza
- (c) Las demás promesas se cancelan de inmediato

**278** Si `pedirB(a)` necesita el resultado de `pedirA()`, ¿cómo deben esperarse esas operaciones?

- (a) Iniciar ambas dentro de `Promise.all` sin pasar el resultado
- (b) Usar `forEach` para que B lea el futuro valor de A
- (c) Primero esperar A y luego iniciar B con ese resultado

**279** Tres pedidos independientes tardan un segundo cada uno. ¿Cuánto tardan aproximadamente en serie y juntos con `Promise.all`?

- (a) Tres segundos en serie y uno juntos
- (b) Un segundo en serie y tres juntos
- (c) Tres segundos en ambos casos

**280** Dentro de un `try`, se hace `await` de una promesa que se rechaza. ¿Qué ocurre?

- ☐ a `await` la convierte en `undefined`
- ☐ b El `catch` la recibe como excepción
- ☐ c El error escapa del `try` por ser asíncronico

**281** Se escribe `await` dentro de una función común y da error. ¿Qué falta?

- ☐ a Declararla `async`
- ☐ b Marcarla como `function*`
- ☐ c Agregar `Promise.all` fuera de la función

## 17 Gestión de archivos con Node.js

282–312

**282** ¿Qué secuencia describe las capas que hay entre un archivo guardado y los datos que usa un programa?

- ☐ a Extensión, clase CSS y DOM
- ☐ b Nombre, variable global y objeto JSON
- ☐ c Ruta, bytes, codificación, texto y formato

**283** ¿Qué garantiza que un archivo se llame `datos.json`?

- ☐ a Que su contenido es JSON válido
- ☐ b Que `readFile` lo devuelve ya parseado como objeto
- ☐ c Nada: la extensión es solo una pista

**284** ¿Qué entorno aporta módulos como `node:fs/promises` para acceder a archivos?

- ☐ a Node.js
- ☐ b El núcleo ECMAScript sin entorno
- ☐ c CSS

**285** ¿Qué módulo de Node para archivos se usa directamente con `async` y `await`?

- ☐ a `readFileSync`
- ☐ b `node:fs/promises`
- ☐ c Un atributo HTML

**286** ¿Qué problema causa usar `readFileSync` en un servidor que atiende muchas solicitudes?

- ☐ a Devuelve una promesa que nadie espera
- ☐ b Lee el archivo con otra codificación
- ☐ c Bloquea el hilo y demora a todas las demás solicitudes

**287** ¿Qué extensión de archivo hace que Node lo trate como módulo ES sin configurar nada en `package.json`?

- ☐ a `.mjs`
- ☐ b `.css`
- ☐ c `.json`

**288** `readFile("datos/entrada.txt")` usa una ruta relativa. ¿Desde qué carpeta la resuelve Node?

- ☐ a El directorio del módulo
- ☐ b `process.cwd()`
- ☐ c La carpeta raíz del sistema

**289** En un módulo ES, ¿qué dato permite armar una ruta relativa a la carpeta del propio archivo?

- ☐ a `import.meta.url`
- ☐ b `process.pid`
- ☐ c `Buffer.byteLength`

**290** ¿Qué función une partes de una ruta sin tener que escribir las barras a mano?

- ☐ a `String.split`
- ☐ b `JSON.parse`
- ☐ c `path.join`

**291** ¿Qué logra `mkdir("salidas", { recursive: true })`?

- ☐ a Sobrescribir cualquier archivo llamado `salidas`
- ☐ b Crear también directorios padres faltantes
- ☐ c Vaciar `salidas` si ya existe

**292** ¿Qué hace `writeFile` por defecto si el archivo destino ya existe?

- ☐ a Lo reemplaza
- ☐ b Falla con `EEXIST`
- ☐ c Agrega al final

**293** ¿Qué flag de `writeFile` hace que falle si el archivo ya existe, en lugar de reemplazarlo?

- ☐ a `w`
- ☐ b `wx`
- ☐ c `a+`

**294** ¿Qué operación añade texto al final sin borrar el contenido previo?

- ☐ a `writeFile` con flag `w`
- ☐ b `unlink`
- ☐ c `appendFile`

**295** ¿Qué devuelve `readFile` cuando no se indica codificación?

- ☐ a Un string en UTF-8
- ☐ b Un objeto JSON parseado
- ☐ c Un Buffer de bytes

**296** ¿Cuántos bytes ocupa el emoji 😊 en UTF-8, según `Buffer.byteLength`?

- ☐ a 2 bytes
- ☐ b 4 bytes
- ☐ c 1 byte

**297** Un `TextDecoder` creado con `{ fatal: true }` recibe bytes que no son UTF-8 válido. ¿Qué hace?

- ☐ a Lanza un error en lugar de insertar un reemplazo
- ☐ b Inserta el carácter de reemplazo
- ☐ c Omite los bytes inválidos

**298** Al leer texto creado en distintos sistemas, ¿qué dos convenciones de fin de línea conviene contemplar?

- ☐ a LF y CRLF
- ☐ b CR y TAB
- ☐ c LF y NUL

**299** Al reemplazar un archivo importante, ¿qué técnica evita que quede a medio escribir si el programa se corta?

- ☐ a Borrar el destino y escribirlo de nuevo
- ☐ b Escribir con el flag `a` sobre el destino
- ☐ c Escribir un archivo temporal y después renombrarlo

**300** ¿Qué información agrega `readdir` cuando se usa con `{ withFileTypes: true }`?

- ☐ a El contenido de cada archivo listado
- ☐ b Si cada entrada es archivo o carpeta
- ☐ c El tamaño y la fecha de cada entrada

**301** ¿Qué diferencia hay entre `stat` y `lstat` cuando la ruta es un enlace simbólico?

- ☐ a `stat` sigue el enlace; `lstat` no
- ☐ b `lstat` sigue el enlace; `stat` no
- ☐ c `lstat` es la versión síncrona de `stat`

**302** Un programa quiere leer un archivo solo si existe. ¿Qué forma evita el problema de que el archivo cambie entre la comprobación y la lectura?

- ☐ a Llamar `access` y asumir que nada cambiará
- ☐ b Comparar solo la extensión
- ☐ c Intentar `readFile` y manejar `ENOENT`

**303** ¿Cómo se procesa línea por línea un archivo de texto muy grande sin cargarlo entero en memoria?

- ☐ a `readFile` y después `split` por saltos
- ☐ b `createReadStream` combinado con `readline`
- ☐ c `readFileSync` con `encoding utf8`

**304** Al leer un archivo con un stream, ¿por qué no conviene tratar cada fragmento (chunk) como una línea?

- ☐ a Los chunks llegan desordenados
- ☐ b Un chunk puede cortar una línea
- ☐ c Un chunk trae solo un carácter

**305** ¿Qué resuelve `pipeline` al conectar un stream de lectura con uno de escritura?

- ☐ a Manejo de errores, cierre y contrapresión
- ☐ b Lectura y escritura en hilos separados del sistema
- ☐ c Carga completa previa para evitar cortes

**306** Un programa abre un archivo con `open` y obtiene un `FileHandle`. ¿Dónde conviene cerrarlo para que se cierre aunque falle la lectura?

- ☐ a Solo después de un éxito
- ☐ b Node lo cierra solo al terminar
- ☐ c En `finally`

**307** Después de `JSON.parse`, ¿qué falta hacer antes de usar los datos en la aplicación?

- ☐ a Validar estructura, tipos y rangos
- ☐ b Nada, si `JSON.parse` no lanzó
- ☐ c Volver a serializar con `stringify`

**308** ¿Por qué `línea.split(",")` no basta para cualquier CSV?

- ☐ a `split` no acepta una coma como separador
- ☐ b `split` pierde el último campo de la línea
- ☐ c Puede haber comas dentro de un campo entre comillas

**309** Al leer un archivo, ¿qué códigos de error distinguen la falta de permisos de una ruta inexistente?

- ☐ a `ENOENT` o `EEXIST`, frente a `EACCES`
- ☐ b `EACCES` o `EPERM`, frente a `ENOENT`
- ☐ c `EMFILE` o `EXDEV`, frente a `ENOTDIR`

**310** Si `rename` falla con `EXDEV` al mover un archivo, ¿qué procedimiento permite completar el movimiento?

- ☐ a Copiar, verificar la copia y borrar el origen
- ☐ b Borrar el origen primero y después copiar el archivo
- ☐ c Reintentar `rename` hasta que funcione

**311** Si procesar miles de rutas con `Promise.all` causa `EMFILE`, ¿qué enfoque equilibra recursos y rendimiento?

- ☐ a Leer todas las rutas en serie, de a una
- ☐ b Limitar cuántas operaciones corren al mismo tiempo con un pool
- ☐ c Convertir todas las rutas a URLs antes de leer

**312** Antes de usar `rm` con `recursive: true`, ¿qué debe comprobarse sobre la ruta?

- ☐ a Que tenga una extensión conocida
- ☐ b Que sea relativa, así no necesita validarse
- ☐ c Que esté dentro de una raíz permitida

## 18 Desarrollo web: del documento a una aplicación del clima

313–362

**313** ¿Cuál es la relación entre Internet y la Web?

- ☐ a La Web funciona sobre Internet
- ☐ b Son dos nombres de la misma red
- ☐ c Internet funciona sobre la Web

**314** En `https://ejemplo.com/apuntes/html.html`, ¿qué representa `/apuntes/html.html`?

- ☐ a El dominio
- ☐ b La ruta
- ☐ c La consulta

**315** ¿Qué agrega escribir `<h1>El tiempo</h1>` en lugar de poner el texto sin etiqueta?

- ☐ a Solo lo muestra más grande
- ☐ b Lo convierte en un enlace
- ☐ c Le da significado de título

**316** ¿Con qué criterio se elige entre `h1` y `h2` para un título?

- ☐ a Por el tamaño de letra que se quiere
- ☐ b Por la importancia visual del texto
- ☐ c Por su nivel dentro de la estructura del documento

**317** ¿Qué diferencia práctica hay entre `class` e `id` en HTML?

- ☐ a El `id` se reutiliza; la clase es única
- ☐ b La clase se reutiliza; el `id` es único
- ☐ c Solo el `id` puede usarse desde CSS

**318** Si `strong` está dentro de `p` y `p` dentro de `section`, ¿qué relación tiene `strong` con `section`?

- ☐ a Es descendiente de `section`
- ☐ b Es hijo directo de `section`
- ☐ c Es padre de `section`

**319** ¿Cuál de estos elementos no necesita etiqueta de cierre en HTML?

- ☐ a `p`
- ☐ b `section`
- ☐ c `input`

**320** ¿Qué sucede con `<button disabled="false">` en HTML?

- ☐ a Queda habilitado
- ☐ b Queda deshabilitado
- ☐ c Depende del navegador

**321** ¿Para qué sirve `<!doctype html>` al inicio del documento?

- ☐ a Activa el modo de estándares en el navegador
- ☐ b Declara el idioma del documento
- ☐ c Carga la hoja de estilos base

**322** ¿Qué elemento semántico identifica el contenido principal de una página?

- ☐ a span
- ☐ b link
- ☐ c main

**323** ¿Qué debe describir el atributo `alt` de una imagen que aporta información?

- ☐ a El título que aparece al pasar el mouse
- ☐ b Lo que la imagen aporta si no se ve
- ☐ c El nombre del archivo de la imagen

**324** ¿Cuándo corresponde usar una tabla (`table`) en HTML?

- ☐ a Cuando los datos son filas y columnas
- ☐ b Para ubicar el menú junto al contenido
- ☐ c Para alinear campos de un formulario

**325** ¿Qué debe coincidir para que `<label for="ciudad">` señale un input?

- ☐ a El valor for y el id del input
- ☐ b El valor for y su class
- ☐ c El texto de label y el placeholder

**326** ¿Qué atributo identifica un campo en el envío nativo de un formulario?

- ☐ a id
- ☐ b name
- ☐ c class

**327** ¿Qué ventaja tiene un botón `submit` real frente a un `div` programado para funcionar como botón?

- ☐ a Validación del servidor sin programarla
- ☐ b Conversión automática a JSON
- ☐ c Envío con Enter y acceso por teclado

**328** ¿Qué limitación tiene el atributo `required` en un campo de texto?

- ☐ a No valida que el texto tenga sentido
- ☐ b Rechaza cadenas de solo espacios
- ☐ c Valida también en el servidor

**329** ¿Qué elemento HTML enlaza una hoja CSS externa desde head?

- ☐ a `<script src="styles.css" type="text/css">`
- ☐ b `<link rel="stylesheet" href="styles.css">`
- ☐ c `<style src="styles.css">`

**330** ¿Qué selecciona `.tarjeta` en CSS?

- ☐ a El elemento con id tarjeta
- ☐ b Los elementos `<tarjeta>` del documento
- ☐ c Elementos con la clase tarjeta

**331** ¿Qué selecciona `.tarjeta.destacado`?

- ☐ a Un descendiente destacado de tarjeta
- ☐ b Elementos con cualquiera de las dos clases
- ☐ c Un mismo elemento que tiene ambas clases

**332** ¿Qué selecciona `.tarjeta p`?

- ☐ a Solo párrafos que también tienen clase tarjeta
- ☐ b Párrafos descendientes de un elemento tarjeta
- ☐ c Todas las tarjetas que contienen cualquier texto

**333** Tres reglas CSS asignan un color distinto al mismo párrafo, con los selectores `#estado`, `.aviso` y `p`. ¿Cuál tiene mayor especificidad?

- ☐ a `#estado`
- ☐ b `.aviso`
- ☐ c `p`

**334** Dos reglas CSS asignan la misma propiedad al mismo elemento, con igual especificidad. ¿Cuál se aplica?

- ☐ a La que aparece después
- ☐ b La que tiene el selector más corto
- ☐ c La que aparece primero

**335** Si se define en `body`, ¿qué propiedad heredan los párrafos que no tienen un valor propio?

- ☐ a margin
- ☐ b border-radius
- ☐ c font-family

**336** ¿Respecto de qué se calcula `1.5rem`?

- ☐ a Del ancho del viewport
- ☐ b Del tamaño de fuente del elemento raíz
- ☐ c Del tamaño de fuente del elemento padre

**337** Con width 300px, padding 20px por lado y border 2px por lado en content-box, ¿cuál es el ancho hasta el borde?

- ☐ a 300px
- ☐ b 344px
- ☐ c 324px

**338** Con box-sizing border-box y width 300px, ¿qué incluye ese ancho?

- ☐ a Contenido más márgenes exteriores
- ☐ b Solo contenido
- ☐ c Contenido, padding y bordes

**339** ¿Qué diferencia hay entre `display none` y `visibility hidden`?

- ☐ a hidden conserva el espacio; none no
- ☐ b none conserva el espacio; hidden no
- ☐ c Ninguno conserva el espacio ocupado

**340** ¿Por qué conviene que el estilo `:focus-visible` se vea con claridad?

- ☐ a Muestra qué control tiene el foco
- ☐ b Resalta el control bajo el mouse
- ☐ c Indica qué campos son obligatorios

**341** ¿Qué consigue `var(--acento)` en una regla CSS?

- ☐ a Lee una variable de JavaScript global
- ☐ b Usa el valor de la propiedad `--acento`
- ☐ c Define la propiedad `--acento`

**342** En una fila flex con dirección row, ¿qué propiedad distribuye el espacio sobrante en el eje principal?

- ☐ a align-items
- ☐ b flex-direction
- ☐ c justify-content

**343** ¿Qué expresa `grid-template-columns: repeat(3, 1fr)` ?

- ☐ a Tres filas de igual altura
- ☐ b Tres columnas de 1px cada una
- ☐ c Tres columnas que reparten el espacio en partes iguales

**344** Un diseño muestra los elementos apilados, pensado primero para móvil. ¿Qué hace una media query con `min-width: 40rem` ?

- ☐ a Aplica desde ese ancho en adelante
- ☐ b Aplica por debajo de ese ancho
- ☐ c Fija el ancho de la página en 40rem

**345** ¿Quién aporta el objeto `document` a JavaScript en una página?

- ☐ a La sintaxis básica ECMAScript
- ☐ b El navegador
- ☐ c La hoja CSS

**346** ¿Qué efecto tiene el atributo `defer` en `<script src="app.js" defer>`, ubicado en el `head` ?

- ☐ a Lo ejecuta apenas termina de descargarse
- ☐ b Lo descarga después de analizar el HTML
- ☐ c Lo ejecuta cuando el navegador terminó de analizar el HTML

**347** ¿Por qué lo que muestra el panel Elementos de las herramientas del navegador puede diferir del archivo HTML original?

- ☐ a Elementos muestra una versión minificada
- ☐ b El DOM se corrige y JavaScript lo cambia
- ☐ c CSS reescribe el HTML al aplicar estilos

**348** ¿Qué devuelve `document.querySelector("#estado")` si no encuentra el elemento?

- ☐ a null
- ☐ b Un elemento vacío
- ☐ c undefined

**349** ¿Qué propiedad conviene usar para mostrar en la página un nombre que llegó de una API?

- ☐ a innerHTML sin validación
- ☐ b textContent
- ☐ c outerHTML con la respuesta completa

**350** ¿Por qué se escribe `boton.addEventListener("click", manejar)` y no `boton.addEventListener("click", manejar())` ?

- ☐ a Debe devolver un elemento al registrarse
- ☐ b El navegador no acepta callbacks
- ☐ c Debe ejecutarse cuando ocurra el evento

**351** ¿Qué hace `evento.preventDefault()` en un submit cancelable?

- ☐ a Cancela el comportamiento nativo de envío
- ☐ b Detiene la propagación del evento
- ☐ c Vacía los campos del formulario

**352** ¿Qué distingue `evento.target` de `evento.currentTarget` ?

- ☐ a target es el origen; currentTarget, el del listener
- ☐ b target es el del listener; currentTarget, el origen
- ☐ c Son el mismo elemento en todos los casos

**353** Una app del clima busca ciudades por nombre y muestra una lista de coincidencias para que la persona elija una. ¿Por qué no usa directamente la primera?

- ☐ a Porque la API exige elegir una por país
- ☐ b Para consultar las coordenadas de la ciudad que se quiso buscar
- ☐ c Para mostrar el clima de todas a la vez

**354** Una app arma la URL de búsqueda con el nombre de una ciudad que tiene espacios y tildes. ¿Qué aporta usar `URLSearchParams` ?

- ☐ a Traduce el nombre de la ciudad al inglés
- ☐ b Garantiza que la red responda
- ☐ c Codifica correctamente los valores de la query

**355** En una app del clima, al empezar una búsqueda nueva se oculta el resultado anterior y se borra la lista de ciudades. ¿Para qué?

- ☐ a Para liberar la memoria que usó el navegador
- ☐ b Para que la API no reciba dos pedidos
- ☐ c Para no mostrar datos viejos como actuales

**356** Una app valida la temperatura con `Number.isFinite(temperatura)` antes de mostrarla. ¿Qué ventaja tiene esto cuando la temperatura es 0 °C?

- ☐ a Descarta 0 por ser un valor falso
- ☐ b Acepta 0 como temperatura válida
- ☐ c Rechaza temperaturas bajo cero

**357** Una API entrega tiempo Unix en segundos. ¿Qué necesita `new Date` para ese instante?

- ☐ a Milisegundos, por lo que se multiplica por 1000
- ☐ b Segundos sin conversión
- ☐ c Una cadena de zona horaria como único argumento

**358** ¿Qué permite `<meta name="viewport" content="width=device-width, initial-scale=1.0">` en una página adaptable?

- ☐ a Que el navegador genere las media queries
- ☐ b Que la página funcione sin CSS
- ☐ c Que el ancho de diseño acompañe al dispositivo móvil

**359** Dentro de un formulario, ¿qué tipo de botón se usa para una acción que no debe enviarlo?

- ☐ a `type="submit"`
- ☐ b `type="button"`
- ☐ c `type="search"`

**360** Si una imagen es puramente decorativa, ¿qué valor de `alt` indica que no aporta contenido?

- ☐ a `alt=""`
- ☐ b Omitir el atributo alt
- ☐ c `alt="imagen decorativa"`

**361** En la cascada de CSS, ¿qué modifica `!important` ?

- ☐ a El número de identificadores en el selector
- ☐ b El orden físico de las reglas en el archivo
- ☐ c La etapa de importancia de la declaración

**362** Una app muestra `Failed to fetch`. ¿Qué panel del navegador ayuda a revisar solicitudes, estados y tiempos?

- ☐ a Elementos o Elements
- ☐ b Red o Network
- ☐ c Aplicación o Application

## 19 Arrow.js: del contador al inventario

363–374

- 363** En Arrow.js, ¿qué hace `reactive({ nombre: "Tornillos", cantidad: 3 })`?
- (a) Hace observables sus propiedades
  - (b) Guarda el objeto en localStorage
  - (c) Convierte el objeto en HTML
- 
- 364** En una plantilla de Arrow.js, ¿por qué se escribe `${() => producto.cantidad}` en lugar de `${producto.cantidad}`?
- (a) Así Arrow sabe qué actualizar al cambiar
  - (b) Así la plantilla se evalúa una sola vez
  - (c) Así cantidad se convierte en texto
- 
- 365** En un inventario hecho con Arrow.js, `Contador` es un componente que muestra la tarjeta de un producto. ¿Qué recibe y qué devuelve?
- (a) Recibe una plantilla; devuelve un producto
  - (b) Recibe un producto; devuelve una plantilla
  - (c) Recibe un id; devuelve HTML en texto
- 
- 366** En Arrow.js, ¿por qué se escribe `Contador(p).key(p.id)` al mostrar una lista de productos?
- (a) Ordena la lista por id
  - (b) Reinicia cada contador al filtrar la lista
  - (c) Identifica cada producto aunque se mueva
- 
- 367** En un buscador hecho con Arrow.js, el campo tiene `.value="${() => estado.búsqueda}"` y `@input="${e => estado.búsqueda = e.target.value}"`. ¿Qué hace cada uno?
- (a) `.value` muestra el estado; `@input` lo actualiza
  - (b) `@input` muestra el estado; `.value` lo actualiza
  - (c) Ambos actualizan el estado al escribir
- 
- 368** En un inventario hecho con Arrow.js, un producto se oculta al filtrar por búsqueda y reaparece al borrar el texto. ¿Por qué conserva su cantidad?
- (a) Cada búsqueda recrea los productos desde cero
  - (b) `filter` devuelve los mismos objetos del estado
  - (c) Arrow guarda la cantidad en localStorage
- 
- 369** En un inventario hecho con Arrow.js, el componente `Formulario` crea su propio estado `borrador`. ¿Por qué?
- (a) Para que el inventario cambie en cada tecla
  - (b) Porque Arrow no permite estado fuera de componentes
  - (c) Para no tocar el inventario hasta confirmar
- 
- 370** ¿Por qué se aplica `Number` a la cantidad que se lee de un campo con `e.target.value`?
- (a) Arrow convierte todo número en bigint
  - (b) El input devuelve undefined si está vacío
  - (c) El valor del campo llega como texto
- 
- 371** En Arrow.js, ¿qué hace `estado.productos = estado.productos.filter(p => p.id !== id)`?
- (a) Elimina todos los productos del mismo nombre
  - (b) Reemplaza la lista por una sin ese producto
  - (c) Oculta el producto sin cambiar la lista

- 372** En un inventario de Arrow.js cuyos datos viven solo en memoria, ¿qué pasa con los productos agregados al recargar la página?
- (a) Se pierden y reaparecen los valores iniciales
  - (b) Se restauran desde una base de datos integrada en Arrow
  - (c) Se recuperan desde localStorage

- 373** En un inventario hecho con Arrow.js, ¿cómo entrega el componente `Formulario` los datos de un producto nuevo a la aplicación?
- (a) Modifica directamente `estado.productos` mientras se escribe
  - (b) Envía el borrador al CDN de Arrow.js
  - (c) Llama a `acciones.onAgregar(datos)` al confirmar

- 374** En una tarjeta hecha con Arrow.js, el botón de restar tiene `disabled="${() => producto.cantidad === 0}"`. ¿Qué pasa cuando la cantidad llega a cero?
- (a) Queda deshabilitado mediante una lectura reactiva de la cantidad
  - (b) Se oculta de la tarjeta
  - (c) Sigue restando hasta números negativos

## 20 Cómo consumir una API REST

375–400

- 375** ¿Cuáles de estos programas pueden actuar como cliente HTTP?
- (a) Solo un navegador con interfaz gráfica
  - (b) Un navegador, curl o fetch desde un programa
  - (c) Un servidor web, pero no curl
- 
- 376** HTTP es un protocolo sin estado (stateless). ¿Qué implica eso?
- (a) El servidor recuerda los pedidos anteriores
  - (b) No se pueden usar cookies ni cabeceras
  - (c) Cada pedido debe incluir lo necesario para entenderse solo
- 
- 377** En la URL `https://ejemplo.com/usuarios?id=7#perfil`, ¿qué parte no se envía al servidor?
- (a) El fragmento
  - (b) La ruta
  - (c) La cadena de consulta
- 
- 378** ¿Qué cabecera indica que el cuerpo del pedido está en formato JSON?
- (a) `Accept: application/json`
  - (b) `Content-Type: application/json`
  - (c) `Content-Encoding: application/json`
- 
- 379** En una API REST de usuarios, ¿qué URL representa al usuario con id 7?
- (a) `/obtenerUsuario?id=7` como única forma REST
  - (b) `/users/7`
  - (c) `/users#7`
- 
- 380** ¿Qué método HTTP se usa normalmente para leer un recurso sin modificarlo?
- (a) POST
  - (b) DELETE
  - (c) GET
- 
- 381** ¿Qué diferencia hay entre los métodos `PATCH` y `PUT`?
- (a) `PATCH` cambia campos; `PUT` reemplaza todo
  - (b) `PUT` cambia campos; `PATCH` reemplaza todo
  - (c) `PATCH` crea recursos; `PUT` los modifica

**382** Un `POST` falla por un error de red y se reintenta. ¿Por qué hay que tener cuidado?

- ☐ a POST es idempotente, como PUT
- ☐ b Puede crear el recurso dos veces
- ☐ c El servidor rechaza POST repetidos

**383** ¿Qué significan los códigos HTTP 201 y 204?

- ☐ a 201 para recurso creado; 204 para éxito sin cuerpo
- ☐ b 201 para éxito sin cuerpo; 204 para recurso creado
- ☐ c 201 para redirección; 204 para validación fallida

**384** ¿Qué diferencia hay entre los códigos HTTP 401 y 403?

- ☐ a 401: no hay permiso; 403: falta autenticarse
- ☐ b 401: servidor caído; 403: recurso movido
- ☐ c 401: falta autenticarse; 403: no hay permiso

**385** ¿Qué informa el código HTTP 429?

- ☐ a Tiempo de espera agotado
- ☐ b Cuerpo JSON inválido
- ☐ c Demasiadas solicitudes

**386** ¿Cómo deben escribirse las claves de un objeto en JSON?

- ☐ a Comillas dobles
- ☐ b Comillas simples
- ☐ c Ninguna comilla

**387** ¿Qué operación convierte un objeto JavaScript en texto JSON para enviarlo?

- ☐ a `JSON.parse`
- ☐ b `JSON.stringify`
- ☐ c `Response.json`

**388** `JSONPlaceholder` es una API de práctica. Se crea un recurso con `POST` y después se lo pide con `GET`. ¿Qué pasa?

- ☐ a No lo encuentra: el POST se simula y no se guarda
- ☐ b Lo encuentra con el id devuelto
- ☐ c Lo encuentra hasta que se reinicia

**389** En un archivo `.http` de la extensión REST Client, ¿qué separa un pedido del siguiente?

- ☐ a Una etiqueta HTML
- ☐ b Un archivo distinto por pedido
- ☐ c `###`

**390** ¿Qué opción de `curl` muestra las cabeceras de la respuesta junto con el cuerpo?

- ☐ a `-d`
- ☐ b `-i`
- ☐ c `-o`

**391** ¿Qué espera cada uno de estos pasos: `await fetch(url)` y `await respuesta.json()` ?

- ☐ a Primero el cuerpo; luego su conversión
- ☐ b Primero las cabeceras; luego el cuerpo
- ☐ c Primero la conexión; luego las cabeceras

**392** Después de `await respuesta.json()`, ¿qué indica `respuesta.bodyUsed` ?

- ☐ a true; el cuerpo ya se consumió
- ☐ b false; puede leerse de nuevo sin límite
- ☐ c undefined porque Response no tiene cuerpo

**393** ¿Qué pasa con la promesa de `fetch` si el servidor responde 404?

- ☐ a La promesa se rechaza con un error
- ☐ b Se resuelve y `respuesta.ok` es true
- ☐ c Se resuelve y `respuesta.ok` es false

**394** Una app de consola hecha con `Ink` maneja los estados `"pidiendo"`, `"cargando"`, `"mostrando"` y `"error"`. ¿Qué indica el estado `"cargando"` ?

- ☐ a Que la API ya devolvió el clima
- ☐ b Que la consulta está en curso y aún no hay respuesta
- ☐ c Que la consulta falló y se reintenta

**395** En un pedido HTTP, ¿qué cabecera expresa el formato de respuesta que acepta el cliente?

- ☐ a `Content-Length`
- ☐ b `Location`
- ☐ c `Accept`

**396** Un servidor recibe JSON bien formado, pero un campo no supera la validación. ¿Qué código HTTP representa ese caso?

- ☐ a 422
- ☐ b 304
- ☐ c 201

**397** ¿Qué comunica un estado HTTP 304 `Not Modified` ?

- ☐ a Que el recurso se movió a otra URL
- ☐ b Que el servidor no reconoce el método
- ☐ c Que la copia en caché sigue valiendo

**398** ¿Qué aporta `url.searchParams.set("ciudad", nombre)` al construir una URL?

- ☐ a Codifica el valor para la query, incluso espacios y tildes
- ☐ b Convierte el nombre en coordenadas
- ☐ c Agrega el valor sin codificar a la URL

**399** En la salida de `curl -v`, ¿qué distinguen las líneas que empiezan con `>` de las que empiezan con `<` ?

- ☐ a `>` es lo recibido; `<` lo enviado
- ☐ b `>` es lo enviado; `<` lo recibido
- ☐ c `>` son errores; `<` son avisos

**400** Si una respuesta de `fetch` tiene estado 204, ¿qué debe hacer una función que espera JSON antes de llamar a `respuesta.json()` ?

- ☐ a No llamar a `json()` y devolver null
- ☐ b Llamar a `json()`, que devuelve `{}`
- ☐ c Tratar 204 como un error